

1. Problem Framing

The ask was straightforward: build a tool that takes a job context — role, level, industry, use case — and returns the right SHL assessments for it. The underlying challenge is that SHL's catalogue spans roughly 400+ products across nine categories. No simple keyword filter handles the nuance of, say, differentiating a volume contact-centre hire from a graduate intake programme. Both might use personality tools, but for entirely different reasons and at different stages.

Rather than hard-coding a decision tree (brittle, hard to maintain), the approach was to encode SHL's product logic into a structured system prompt and let an LLM handle the matching. The product catalogue acts as the knowledge base; the model acts as the reasoning engine.

2. Architecture

The app is a single-file Streamlit dashboard. Keeping it in one file was a deliberate choice — easier to hand off, no packaging overhead. Three layers do the work:

Layer	What it does	Technology
UI	Collects role context, renders recommendation cards, manages session history	Streamlit + custom CSS
Prompt layer	Embeds the SHL catalogue as structured domain knowledge; instructs the model to return strict JSON	SYSTEM_PROMPT constant
LLM dispatcher	Routes the call to Anthropic, OpenAI, or Gemini based on sidebar selection	call_llm() function

The multi-provider design came from a practical observation: different teams already have API contracts with different vendors. Locking the tool to Anthropic alone would limit adoption. The provider registry pattern — a single dict driving the dropdown, model list, key placeholder, and CSS badge — means adding a fourth provider is a one-block change.

3. Prompt Engineering

The system prompt is the core IP here. It is structured in two parts: the catalogue and the output contract.

Catalogue section

Each product category (Cognitive, Personality, Behavioral, etc.) is listed with its key differentiators — not just names. For example, Verify G+ is described as adaptive/IRT and remote-compatible, which matters when the use case is volume hiring. This gives the model enough signal to make

context-sensitive choices rather than defaulting to the most well-known tool every time.

Output contract

The model is told to return only valid JSON, with no preamble, in a strict schema: name, type, priority, rationale, measures, stage, duration. The *priority* field (Core / Highly recommended / Supporting) is deliberate — it lets the UI split cards into sections and gives the user a clear starting point rather than an unranked list. Strict JSON output also means the app never needs to parse prose, which is where fragility usually enters. A post-processing step strips any markdown code fences the model occasionally adds despite being instructed not to.

4. Session Management

Streamlit re-executes the entire script on every user interaction. Without careful state management, form values reset on every click. The solution is a single `_init_state()` function called at app startup that sets defaults for every key — but only if the key does not already exist in `st.session_state`. The *if k not in* guard is the critical detail that makes persistence work.

Each successful API call is serialised into a history entry containing the full result dict, timestamp, job context, provider name, and model. The sidebar renders these as a scrollable list capped at 20 entries. Loading a past entry writes all fields back to session state and triggers a rerun — giving the user a one-click restore without any backend storage.

API keys are kept in session state (not persisted to disk or environment). When the user switches provider, the old key is explicitly cleared. This prevents a key from one provider being inadvertently used with another provider's endpoint.

5. Key Design Decisions

Single-file app

Sacrifices modularity for portability. The right call for a tool that needs to be shared and run quickly without a package install step.

Catalogue in the prompt, not a database

SHL's catalogue changes slowly. Embedding it in the system prompt avoids the overhead of a vector store or retrieval layer at this scale. If the catalogue grows significantly, a RAG approach would be the next step.

Structured JSON output

Forces deterministic parsing. The alternative — asking the model to write prose recommendations — produces better-sounding text but breaks the card UI and makes priority sorting impossible.

Optional SDKs for OpenAI / Gemini

Imported with try/except so the app works out of the box with just the Anthropic package. Missing SDKs surface a warning, not a crash.

No authentication layer

The current design is suitable for internal or single-user deployment. Adding multi-user auth (e.g. via Streamlit Cloud's built-in auth or a middleware like streamlit-authenticator) is the obvious production step.

6. If I Had More Time

The two things I'd tackle first are streaming and comparison mode. Right now the app waits for the full response before rendering anything — streaming the JSON incrementally would make it feel significantly faster, especially on larger outputs. Comparison mode would let a user run the same job context across two providers side-by-side, which would be genuinely useful for evaluating model quality on this specific task.

On the product side, the quick-fill templates are useful shortcuts but they pre-fill all five fields at once, which can obscure what the model is actually responding to. A better UX would be progressive disclosure — start with just the job title, and have the app suggest likely levels and industries based on what it knows, letting the user confirm or adjust before the full call is made.

Built with Streamlit · Anthropic Claude · OpenAI GPT · Google Gemini · SHL Product Catalogue